MACHINE LEARNING DRIVEN GAS SENSOR NETWORK FOR ON-SITE AGROCHEMICAL HAZARDOUS SCENT DETECTION

A PROJECT REPORT

Submitted by

JAI AKASH L (312822205020)

UDITH YASHICA M(312822205055)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

AGNI COLLEGE OF TECHNOLOGY

(An Autonomous Institution)

(Affiliated to Anna University,

Chennai)

APRIL 2026

AGNI COLLEGE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report **"MACHINE LEARNING DRIVEN GAS SENSOR NETWORK FOR ON-SITE AGROCHEMICAL HAZARDOUS SCENT DETECTION"** is the bonafide work of **"JAI AKASH L(312822205020), UDITH YASHICA M(312822205055)"** who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
Dr. G. SIMI MARGARAT, M.Tech, Ph.D	S. MANORANJITHAM, M.E
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Department of Information Technology	Department of Information Technology
Agni College of Technology,	Agni College of Technology,
Thalambur, Chennai –600130	Thalambur, Chennai - 600130

Submitted for the project viva held on ……

SIGNATURE OF INTERNAL EXAMINER	SIGNATURE OF EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to the management of and would like to thank our respected Principal, **Dr. SRINIVASAN ALAVANDAR** for his words of inspiration and for providing the necessary facilities to carry out our project work successfully.

We are immensely thankful to **Dr. G. SIMI MARGARAT, M.Tech, Ph.D.,** Associate Professor, Head of the Department, Information Technology, for his words of wisdom and his constant source of inspiration.

I would like to offer our heartfelt thanks to our guide, **MANORANJITHAM S, M.E.,** Assistant Professor, Department of Information Technology, who guided us and provided valuable suggestions to complete our project work successfully.

We extend our warmest thanks to all the faculty members of our department for their assistance, and we also thank all our friends who helped us bring out our project in good shape and form.

Finally, we express our sincere benevolence to our beloved parents for their perpetual encouragement and support in all endeavors.

ABSTRACT

This project presents a machine-learning-based framework for detecting hazardous agrochemical odors using an IoT-enabled gas-sensor network, aimed at providing real-time, on-site monitoring and reducing reliance on laboratory analysis. The system uses an MQ135 gas sensor integrated with a microcontroller to collect air quality data and transmit it to a cloud platform for processing. Data preprocessing techniques such as normalization, feature extraction, and Principal Component Analysis (PCA) are applied to improve data quality and reduce dimensionality. Multiple supervised learning models, including Support Vector Regression (SVR), Backpropagation Neural Network (BPNN), Random Forest Regression (RFR), and Classification and Regression Tree (CART), are implemented and evaluated using Mean Squared Error (MSE) and Coefficient of Determination (R^2). Results show that SVR and BPNN provide better accuracy and generalization compared to other models. The system classifies gas levels into SAFE, WARNING, and DANGER categories and provides outputs through graphical visualization, voice alerts, and siren warnings. Overall, the proposed framework offers a cost-effective, scalable, and efficient solution for environmental monitoring, enhancing safety and supporting informed decision-making in agricultural environments.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	**I**
	LIST OF FIGURES	**IV**
	LIST OF ABBREVIATIONS	**V**
1	**INTRODUCTION**	**1**
	1.1 INTRODUCTION	1
	1.2 OBJECTIVE	2
2	**LITERATURE SURVEY**	3
3	**SYSTEM ANALYSIS**	**6**
	3.1 EXISTING SYSTEM	6
	3.2 DISADVANTAGEs	6
	3.3 PROPOSED SYSTEM	7
	3.4 ADVANTAGES	7
4	**SYSTEM REQUIREMENTS**	**8**
	4.1 FUNCTIONAL REQUIREMENTS	8
	4.2 NON-FUNCTIONAL REQUIREMENTS	8
	4.3 ENVIRONMENT REQUIREMENTS	9
5	**SYSTEM DESIGN**	**10**
	5.1 DATA FLOW DIAGRAM	10
	5.2 WORK FLOW DIAGRAM	11
	5.3 USE CASE DIAGRAM	11
	5.4 CLASS DIAGRAM	12
	5.5ACTIVITY DIAGRAM	13
	5.6 SEQUENCE DIAGRAM	14
	5.7 ER DIAGRAM	15
	5.8 COLLABORATION DIAGRAM	16

6 SYSTEM IMPLEMENTATION 17
6.1 INTRODUCTION 17
6.2 HARDWARE IMPLEMENTATION 17
6.3 IOT COMMUNICATION 18
6.4 CLOUD INTEGRATION 18
6.5 PYTHON-BASED PROCESSING 19
6.7 GAS CLASSIFICATION 19
6.9 ALERT SYSTEM 19
6.10 USER INTERFACE 21
6.11 SYSTEM INTEGRATION SUMMARY 21
7 CODING AND TESTING 22
7.1 INTRODUCTION 22
7.2 EMBEDDED SYSTEM CODING (ESP8266) 22
7.3 PYTHON CODING 23
7.4 MACHINE LEARNING CODE 23
7.5 OUTPUT GENERATION CODE 23
7.6 SYSTEM TESTING METHODOLOGY 24
7.7 PERFORMANCE EVALUATION 24
7.8 TESTING RESULT 24
7.9 ERROR HANDLING 25
8 SYSTEM DESCRIPTION 26
8.1 ARDUINO IDE 26
8.2 ANACONDA NAVIGATOR 26
8.3 PYTHON 26
8.4 THINGSPEAK CLOUD PLATFORM 26
8.5 LIBRARIES USED 26
9 CODING 28
9.1 ESP8266 (ARDUINO CODE) 28

9.2 PYTHON TRAINING CODE 29
9.3 PYTHON LIVE PREDICTION 30

10 RESULT AND DISCUSSION 38
10.1 RESULTS 38
10.2 CONCLUSION 39
10.3 FUTURE WORK 39
REFERENCE 41
IEEE PAPER 49

LIST OF FIGURES

FIGURE NO	TITLE	PAGE
5.1	Data Flow Diagram	10
5.2	Workflow Diagram	11
5.3	Use Case Diagram	12
5.4	Class Diagram	13
5.5	Activity Diagram	14
5.6	Sequence Diagram	15
5.7	ER Diagram	16
5.8	Collaboration Diagram	17
6.3	Proposed Methodology	18
9.1	Screenshots Arduino Output Page	33
9.2	Thinkspeak Monitoring Page	34
9.3	System Setup	35
9.4	Output Page	36
9.5	Graph Monitoring Page	37

LIST OF ABBREVIATIONS

SVR	SUPPORT VECTOR REGRESSION
PCA	PRINCIPAL COMPONENT ANALYSIS
MSE	MEAN SQUARED ERROR
COD	COEFFICIENT OF DETERMINATION
IOT	INTERNET OF THINGS
GC-MS	GAS CHROMATOGRAPHY-MASS SPECTROMETRY

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Agriculture plays a vital role in the global economy, and the use of agrochemicals such as pesticides, herbicides, and fertilizers has become essential for improving crop yield. However, improper handling, accidental leakage, or overuse of these chemicals can lead to hazardous environmental and health conditions. Exposure to such toxic gases can affect farmers, nearby communities, and ecosystems. Traditional methods for detecting agrochemical hazards rely on laboratory-based techniques such as Gas Chromatography and Mass Spectrometry. Although these methods provide accurate results, they are expensive, time-consuming, and unsuitable for real-time monitoring. With advancements in sensor technology and machine learning, it is now possible to develop intelligent systems capable of analyzing sensor data and predicting hazardous conditions. Gas sensor networks can continuously monitor environmental conditions and provide real-time data for analysis. This project focuses on developing a machine-learning-based system using Support Vector Regression (SVR) to predict hazard levels from sensor readings. The system aims to provide a fast, reliable, and cost-effective solution for on-site.

1.2 OBJECTIVE

The primary objective of this project is to develop an efficient and reliable system for detecting hazardous agrochemical gases using a machine learning-based approach. The system aims to utilize sensor-derived data to accurately predict the level of hazard present in the environment. A key focus of this work is the implementation of Support Vector Regression (SVR) as the core predictive model, owing to its strong performance in handling nonlinear relationships and small datasets. In addition, appropriate data preprocessing techniques such as normalization and Principal Component Analysis (PCA) are employed to enhance model accuracy and reduce redundancy in the data. The project also seeks to evaluate the performance of the proposed model using standard metrics such as Mean Squared Error (MSE) and Coefficient of Determination (R^2). Ultimately, the objective is to design a cost-effective, scalable, and real-time monitoring solution that can assist in early detection and prevention of hazardous agrochemical exposure.

CHAPTER 2

LITERATURE SURVEY

[1] Project Title: Evaluation of Methods for the Detection of Hazardous Substances in Food Based on Machine Learning

Author: L.-y. Zhu, L. Yan, F. Zhao, X. Guo, D. Xu, J. Lv, L. Ding, N. Niu, J.-q. Qiao, S. Ma, X. Huang, H. Liu, and H.-z. Lian

This study presents a comprehensive evaluation of various machine learning techniques for detecting hazardous substances in food products. With the increasing concerns over food safety and public health, the need for accurate and efficient detection systems has become essential. Traditional laboratory-based methods, although reliable, are often time-consuming, expensive, and require skilled personnel. In contrast, machine learning offers a faster and more scalable solution for analyzing large datasets and identifying potential hazards in food. The authors propose a structured evaluation framework that considers multiple performance metrics such as sensitivity, accuracy, cost-effectiveness, and operational complexity. This framework enables a systematic comparison of different machine learning models to determine their suitability for food safety applications. The study highlights how data-driven approaches can significantly improve the detection of harmful substances such as pesticides, heavy metals, and chemical contaminants. The paper compares several supervised learning models, including Support Vector Regression (SVR), Backpropagation Neural Networks (BPNN), Classification and Regression Trees (CART), and Random Forest Regression (RFR). Each model is evaluated based on its predictive accuracy and ability to handle complex, high-dimensional datasets.

[2] Project Title: Machine Learning-Based Analytical Systems: Food Forensics

Author: R. Ranbir and A. Kumar

This paper explores the application of machine learning techniques in food forensics and analytical systems, focusing on identifying contamination, adulteration, and chemical hazards in food products. The authors emphasize the growing need for advanced analytical approaches due to the increasing complexity of food supply chains and the limitations of traditional laboratory testing methods. Machine learning provides a scalable and efficient alternative for analyzing large datasets generated through chemical and sensor-based techniques. The study highlights how various supervised and unsupervised machine learning models can be applied to detect anomalies and classify food samples based on their chemical composition. Algorithms such as Support Vector Machines (SVM), Decision Trees, and Neural Networks are discussed for their effectiveness in handling high-dimensional data. These techniques enable rapid detection of adulterants and contaminants, improving both speed and accuracy compared to manual inspection methods. In addition, the authors discuss the integration of machine learning with analytical chemistry techniques such as spectroscopy and chromatography. This integration allows automated interpretation of complex datasets, reducing human error and improving reproducibility. The paper also examines the role of feature selection and dimensionality reduction in improving model performance and minimizing computational cost. Furthermore, the research emphasizes the importance of real-time monitoring systems in food safety applications.

[3] Project Title: Machine Learning and Flavoromics-Based Research for Food Quality and Safety

Author: Ray O. Bahado-Singh1, Sangeetha Vishweswaraiah

This research focuses on combining machine learning with flavoromics techniques to improve food quality and safety analysis. Flavoromics involves the study of chemical compounds responsible for taste and aroma, and when integrated with machine learning, it enables more accurate detection of variations in food quality. The authors highlight the importance of understanding complex chemical interactions in food products for better quality assessment.

The study explains how machine learning algorithms can process large volumes of sensor data and chemical attributes to identify patterns and anomalies. Techniques such as Principal Component Analysis (PCA), clustering, and regression models are used to extract meaningful features from complex datasets. These methods help in reducing dimensionality and improving the efficiency of predictive models.

Additionally, the paper discusses the role of preprocessing techniques such as normalization, noise reduction, and feature extraction in enhancing model accuracy. Proper preprocessing ensures that irrelevant or redundant data does not affect the performance of machine learning models. This step is crucial in achieving reliable and consistent results.

The authors also demonstrate the use of sensor-based systems combined with machine learning for real-time monitoring of food quality. These systems can detect subtle changes in flavor profiles, which may indicate spoilage or contamination.

[4] Project Title: Advancements in Intelligent Sensing Technologies for Environmental and Food Safety Monitoring

Author: W. Jiang, H. Sun, and Y. Zheng

This paper discusses recent advancements in intelligent sensing technologies combined with machine learning for environmental and food safety monitoring. The authors highlight the increasing demand for real-time monitoring systems capable of detecting hazardous conditions efficiently. Traditional methods are often time-consuming and require extensive laboratory analysis, making them less suitable for rapid detection.

The study explains how modern sensor systems can collect real-time data from various environmental and food-related parameters. These sensors, when integrated with machine learning algorithms, can analyze data instantly and provide accurate predictions. Techniques such as Artificial Neural Networks (ANN), Random Forest, and Deep Learning models are used to process sensor data and detect anomalies. Furthermore, the paper emphasizes the importance of automated calibration and data preprocessing in maintaining system accuracy. Continuous model retraining is also discussed as a key factor in adapting to changing environmental conditions. This ensures that the system remains reliable over time and can handle variations in data patterns. The research also explores applications of intelligent sensing systems in industrial environments, supply chains, and agricultural monitoring. These systems help in detecting contamination, monitoring storage conditions, and ensuring compliance with safety standards. Real-time alerts and automated decision-making further enhance the effectiveness of these systems. The paper concludes that intelligent sensing technologies play a critical role in modern safety monitoring systems.

[5] Project Title: Applications of Machine Learning in Food Safety and Quality Control: A Review

Author: P. K. Revelou and N. Daniel

This review paper provides a comprehensive overview of various machine learning applications in food safety and quality control. The authors analyze different machine learning techniques used for detecting contaminants, ensuring food quality, and improving monitoring systems. The increasing availability of data and advancements in computational power have made machine learning a valuable tool in this domain. The study discusses commonly used algorithms such as Neural Networks, Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and ensemble methods. These algorithms are effective in classification, regression, and anomaly detection tasks. The paper highlights how these techniques can be applied to detect chemical contaminants, microbial activity, and physical defects in food products. In addition, the importance of interpretability and transparency in machine learning models is emphasized. For regulatory acceptance, it is essential that models provide understandable results. The authors discuss techniques such as feature importance analysis and model visualization to improve transparency. The paper also examines preprocessing techniques such as normalization, data cleaning, and noise reduction. These steps are crucial in improving model performance and ensuring reliable predictions. Proper data handling significantly impacts the accuracy and efficiency of machine learning systems. Finally, the review concludes that machine learning offers a scalable and powerful solution for food safety and quality control. It enables automation, reduces human intervention, and improves detection accuracy.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM:

The existing systems for detecting hazardous agrochemical substances primarily depend on conventional laboratory-based analytical techniques such as Gas Chromatography–Mass Spectrometry (GC-MS) and Liquid Chromatography–Mass Spectrometry (LC-MS). These methods are highly accurate and reliable for identifying chemical compounds; however, they require expensive equipment, specialized laboratory facilities, and trained personnel to operate. In practical scenarios, samples are collected from agricultural fields and transported to laboratories for analysis, which introduces significant delays in obtaining results and makes the process unsuitable for real-time monitoring. Additionally, these systems are not designed for continuous environmental surveillance, as they lack integration with sensor networks and automated data processing techniques. The absence of machine learning-based analysis further limits their ability to handle dynamic environmental conditions and detect hazards instantly. As a result, although existing methods provide precise results, they are not efficient for on-site, real-time, and cost-effective detection of hazardous agrochemical exposure.

3.2 DISADVANTAGES:

I. Existing systems rely on laboratory-based methods, which are expensive and require specialized equipment and skilled personnel.

II. The process of sample collection and laboratory analysis is time-consuming and not suitable for real-time monitoring.

III. These systems do not use sensor networks or machine learning techniques, making them ineffective for continuous monitoring and early hazard detection.

3.3 PROPOSED SYSTEM:

The proposed system aims to develop a machine learning-based solution for real-time detection of hazardous agrochemical gases using sensor data. In this system, gas sensors are used to collect environmental data, which is then preprocessed using techniques such as Min-Max normalization and Principal Component Analysis (PCA) to improve data quality and reduce dimensionality. The processed data is fed into a Support Vector Regression (SVR) model, which predicts the level of hazard based on sensor inputs. The system is designed to provide fast, accurate, and cost-effective detection compared to traditional methods. Additionally, it supports continuous monitoring and can be extended for real-time applications, making it suitable for agricultural environments and safety monitoring systems..

3.4 ADVANTAGES:

I. The proposed system enables real-time detection of hazardous agrochemical gases using sensor data.

II. It provides higher accuracy in prediction by using machine learning techniques such as Support Vector Regression (SVR).

III. The system is cost-effective compared to traditional laboratory-based methods.

IV. It supports continuous monitoring and early hazard detection, improving environmental and human safety

CHAPTER 4

SYSTEM REQUIREMENTS

The system requirements define the necessary hardware and software components required to develop and implement the proposed agrochemical hazard detection system. These requirements ensure the proper functioning and performance of the system. The following are the requirements that are to be discussed.

1. Functional requirements

2. Non-Functional requirements

3. Environment requirements

    A. Hardware requirements

    B. software requirements

4.1 FUNCTIONAL REQUIREMENTS:

The system should be able to collect data from gas sensors, preprocess the data using techniques such as normalization and PCA, and apply the Support Vector Regression (SVR) model to predict hazard levels. It should also display the results for monitoring and support continuous data processing for real-time detection of hazardous agrochemical substances.

4.2 NON-FUNCTIONAL REQUIREMENTS:

The system should provide high accuracy in predicting hazard levels and ensure a fast response time for real-time monitoring. It should be reliable, scalable, and capable of handling continuous data processing without failure.

Additionally, the system should be cost-effective, easy to use, and maintain consistent performance under different environmental conditions.

4.3 ENVIRONMENT REQUIREMENTS:

Software Requirements:

Operating System	: Windows / Linux
Development Tool	: Jupyter Notebook
Language	: Python
Machine Learning Library	: Scikit-learn
Data Processing Libraries	: NumPy, Pandas
Visualization Tool	: Matplotlib

Hardware requirements:

Processor	: Intel i3 or above
Storage	: minimum 100 GB
RAM	: minimum 4 GB
Gas Sensor	: MQ135 Sensor
Microcontroller	: ESP8266/Arduino

CHAPTER 5

SYSTEM DESIGN

System design represents the overall structure and working of the proposed agrochemical hazard detection system. It defines how data is collected, processed, and used to generate predictions. The system design includes various diagrams that illustrate the flow of data, interaction between components, and overall system functionality. The proposed system is designed to collect sensor data, preprocess it, and apply the Support Vector Regression (SVR) model to predict hazard levels in real-time..

5.1 DATA FLOW DIAGRAM:

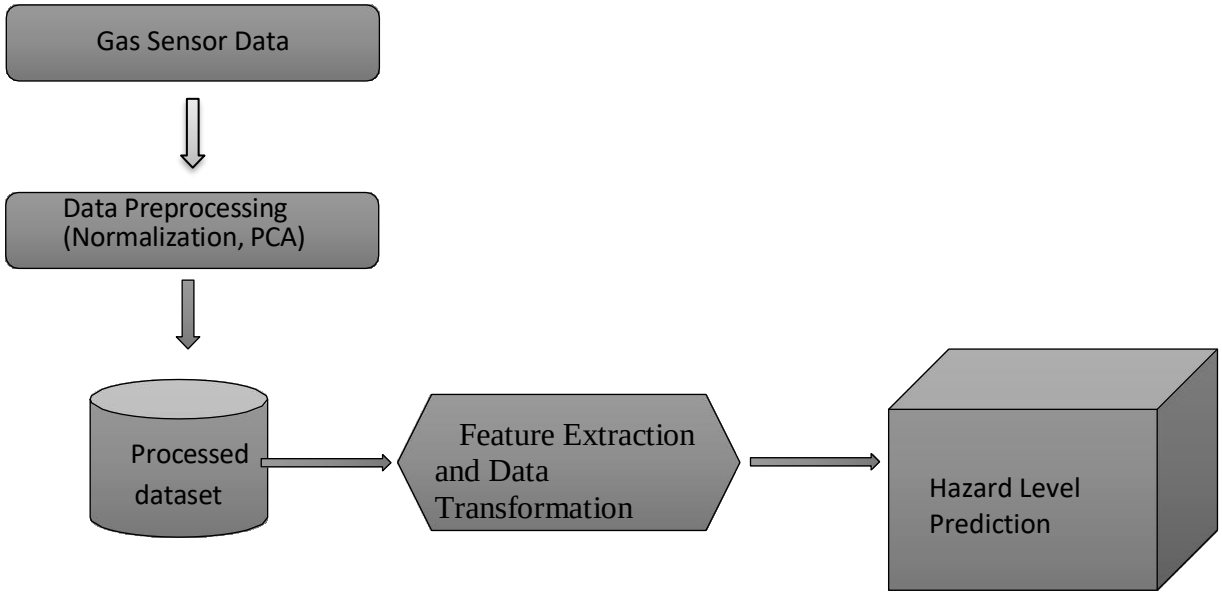


Fig 5.1: Process of data flow diagram

The workflow diagram illustrates the overall working process of the proposed system in a step-by-step manner. Initially, data is collected from gas sensors placed in the environment to monitor agrochemical presence. The collected data is then preprocessed using techniques such as normalization and Principal

Component Analysis (PCA) to improve data quality and reduce dimensionality. The processed data is given as input to the Support Vector Regression (SVR) model, which predicts the hazard level based on sensor readings. Finally, the predicted results are displayed to the user for monitoring and analysis, enabling real-time detection of hazardous conditions.

5.2 WORK FLOW DIAGRAM:

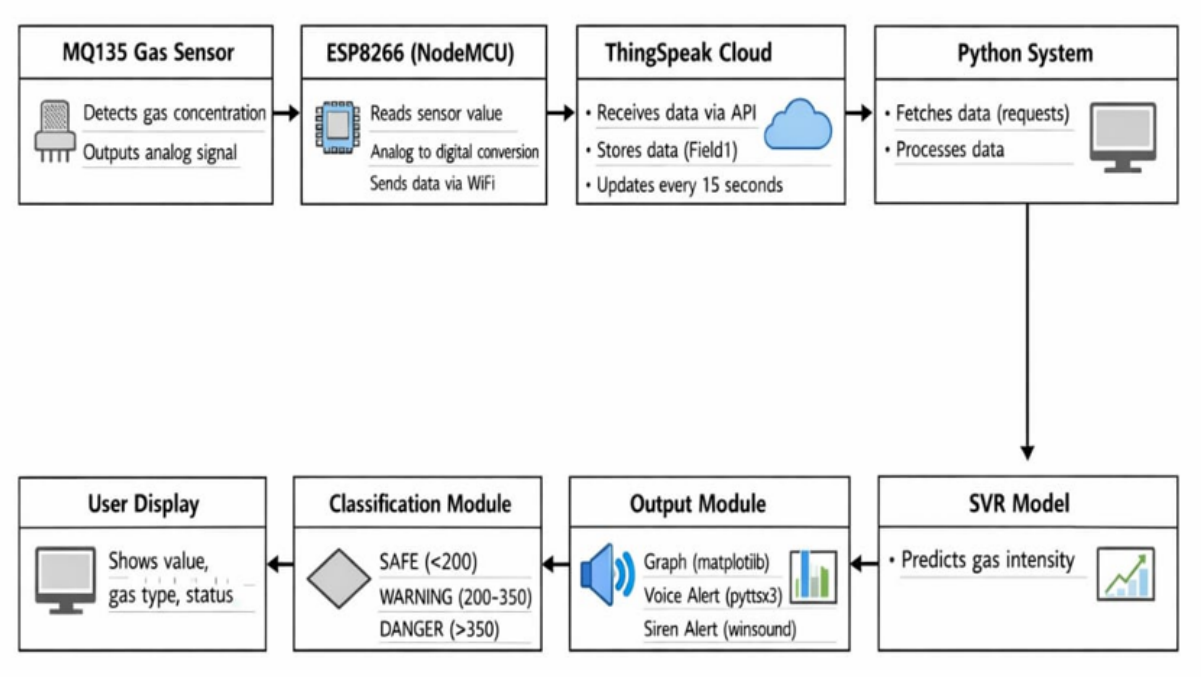


Fig 5.2 : Workflow Diagram

5.3 USECASE DIAGRAM:

Use case diagrams are considered for high level requirement analysis of a system. So when the requirements of a system are analyzed the functionalities are captured in use cases. So, it can say that uses cases are nothing but the system functionalities written in an organized manner.

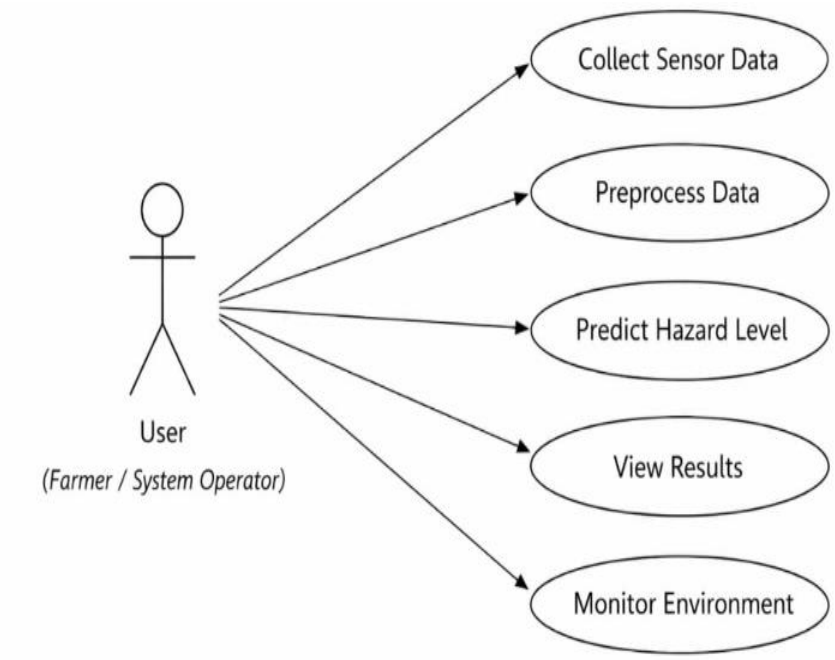


Fig 5.3 : Use case diagram

5.4 CLASS DIAGRAM:

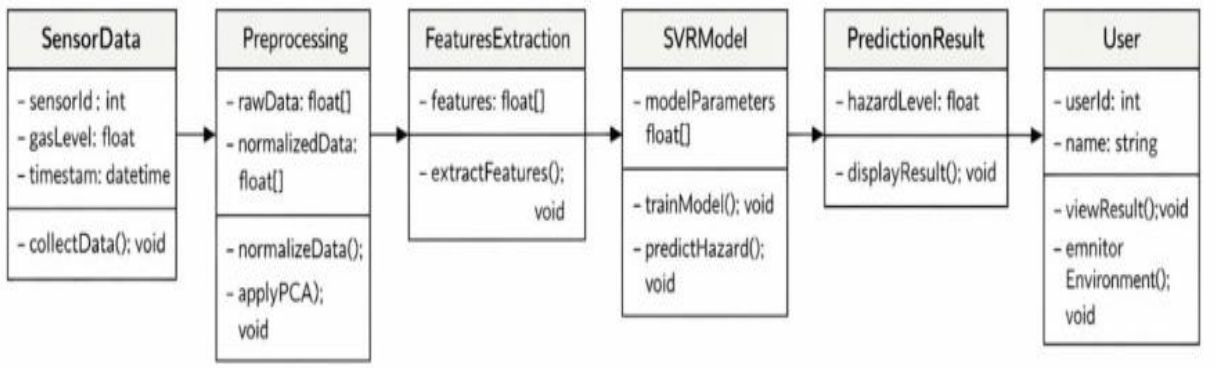


Fig 5.4: Class diagram

Class diagram is basically a graphical representation of the static view of the system and represents different aspects of the application. So a collection of class diagrams represent the whole system. The name of the class diagram should be meaningful to describe the aspect of the system. Each element and their relationships should be identified in advance Responsibility (attributes and

methods) of each class should be clearly identified for each class minimum number of properties should be specified and because, unnecessary properties will make the diagram complicated. Use notes whenever required to describe some aspect of the diagram and at the end of the drawing it should be understandable to the developer/coder. Finally, before making the final version, the diagram should be drawn on plain paper and rework as many times as possible to make it correct.

5.5 ACTIVITY DIAGRAM:

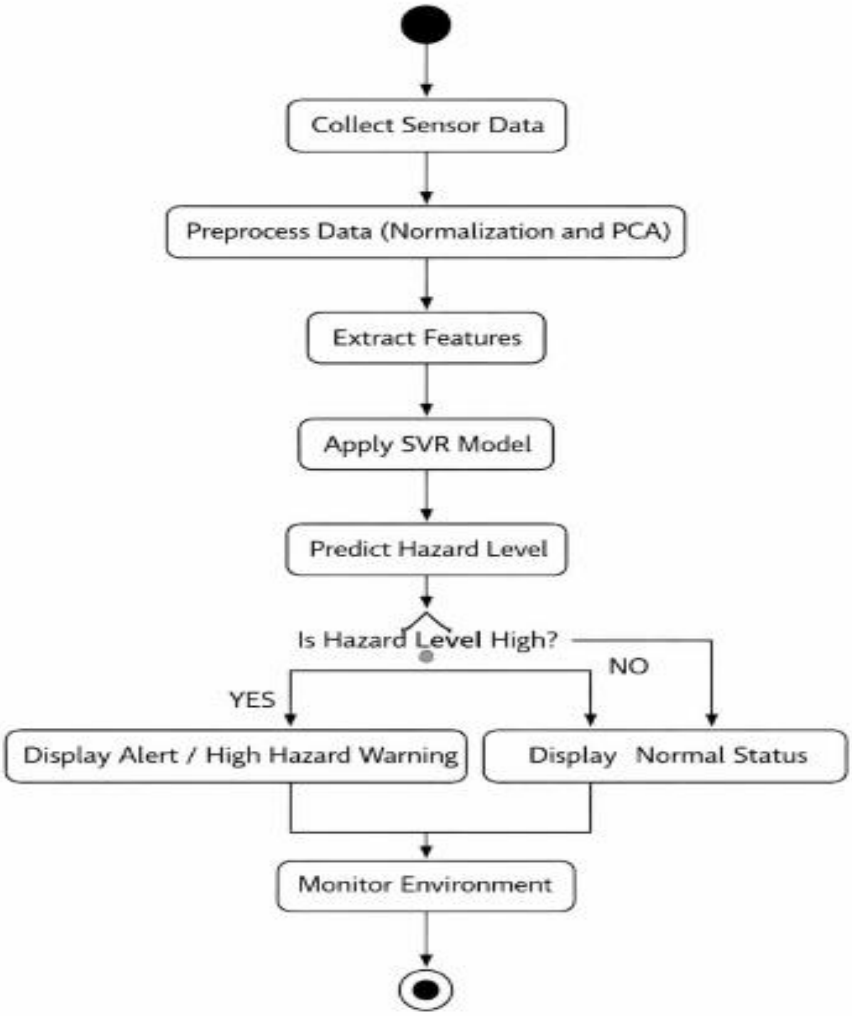


Fig 5.5: Activity diagram

Activity is a particular operation of the system. Activity diagrams are not only used for visualizing dynamic nature of a system but they are also used to construct the executable system by using forward and reverse engineering techniques. The only missing thing in activity diagram is the message part. It does not show any message flow from one activity to another. Activity diagram is some time considered as the flow chart. Although the diagrams looks like a flow chart but it is not. It shows different flow like parallel, branched, concurrent and single.

5.6 SEQUENCE DIAGRAM:

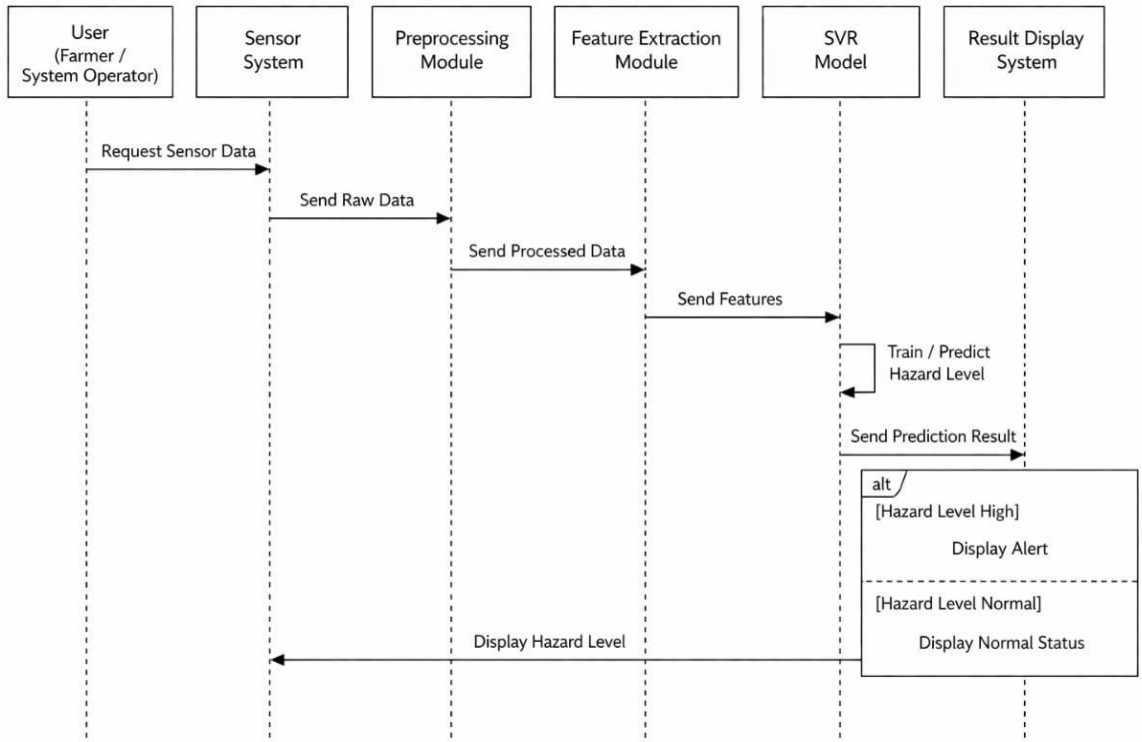


Fig 5.6 : Sequence diagram

Sequence diagrams model the flow of logic within your system in a visual manner, enabling you both to document and validate your logic, and are

commonly used for both analysis and design purposes. Sequence diagrams are the most popular UML artifact for dynamic modelling, which focuses on identifying the behaviour within your system. Other dynamic modelling techniques include activity diagramming, communication diagramming, timing diagramming, and interaction overview diagramming. Sequence diagrams along with class diagrams and physical data models are in my opinion the most important design-level models for modern business application development.

5.7 ER DIAGRAM:

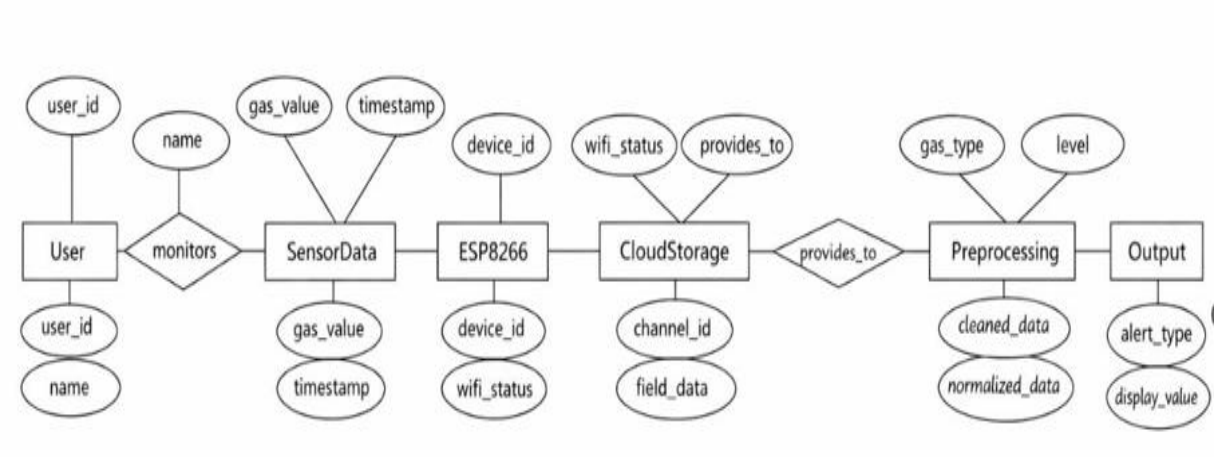


Fig 5.7 : ER diagram

An entity relationship diagram (ERD), also known as an entity relationship model, is a graphical representation of an information system that depicts the relationships among people, objects, places, concepts or events within that system. An ERD is a data modeling technique that can help define business processes and be used as the foundation for a relational database. Entity relationship diagrams provide a visual starting point for database design that can also be used to help determine information system requirements throughout an organization. After a relational database is rolled out, an ERD can still serve as a referral point, should any debugging or business process re-engineering be needed later.

5.8 COLLABORATION DIAGRAM:

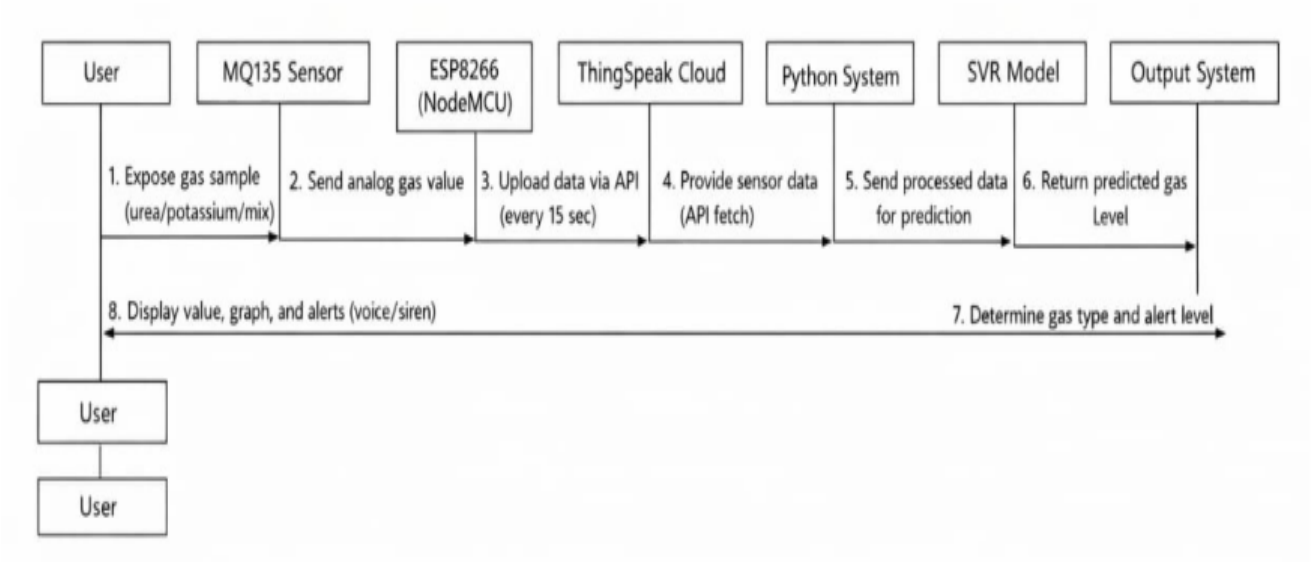


Fig 5.8 : Collaboration diagram

A collaboration diagram show the objects and relationships involved in an interaction, and the sequence of messages exchanged among the objects during the interaction.

The collaboration diagram can be a decomposition of a class, class diagram, or part of a class diagram.it can be the decomposition of a use case, use case diagram, or part of a use case diagram.

The collaboration diagram shows messages being sent between classes and object (instances). A diagram is created for each system operation that relates to the current development cycle (iteration).

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 INTRODUCTION

System implementation describes the practical realization of the proposed gas detection system by integrating hardware components, cloud infrastructure, and software modules into a unified framework. The implementation ensures that the system operates in real time, enabling continuous monitoring, prediction, and alert generation based on gas concentration levels. The system is designed to process sensor data efficiently and provide meaningful output through machine learning techniques.

6.2 HARDWARE IMPLEMENTATION

The hardware implementation involves the MQ135 gas sensor and the ESP8266 (NodeMCU) microcontroller. The MQ135 sensor is used to detect gases present in the environment, particularly those emitted from substances such as urea, potassium, and their mixed combinations. The sensor produces an analog voltage output corresponding to the gas concentration level. The analog output of the MQ135 sensor is connected to the A0 pin of the ESP8266. The microcontroller reads this analog signal and converts it into a digital value using its built-in analog-to-digital converter (ADC). This digital value represents the gas concentration and serves as the primary input for further processing.

6.3 IOT COMMUNICATION IMPLEMENTATION

The ESP8266 microcontroller is configured to connect to a wireless network using predefined credentials such as SSID and password. Once connected, the microcontroller transmits the sensor data to the ThingSpeak cloud platform using HTTP protocol and API keys. Each sensor reading is uploaded to a specific field in the ThingSpeak channel. The data transmission occurs at regular intervals of 15 seconds to ensure real-time monitoring while complying with the platform limitations. The cloud platform stores the data and makes it accessible for further analysis.

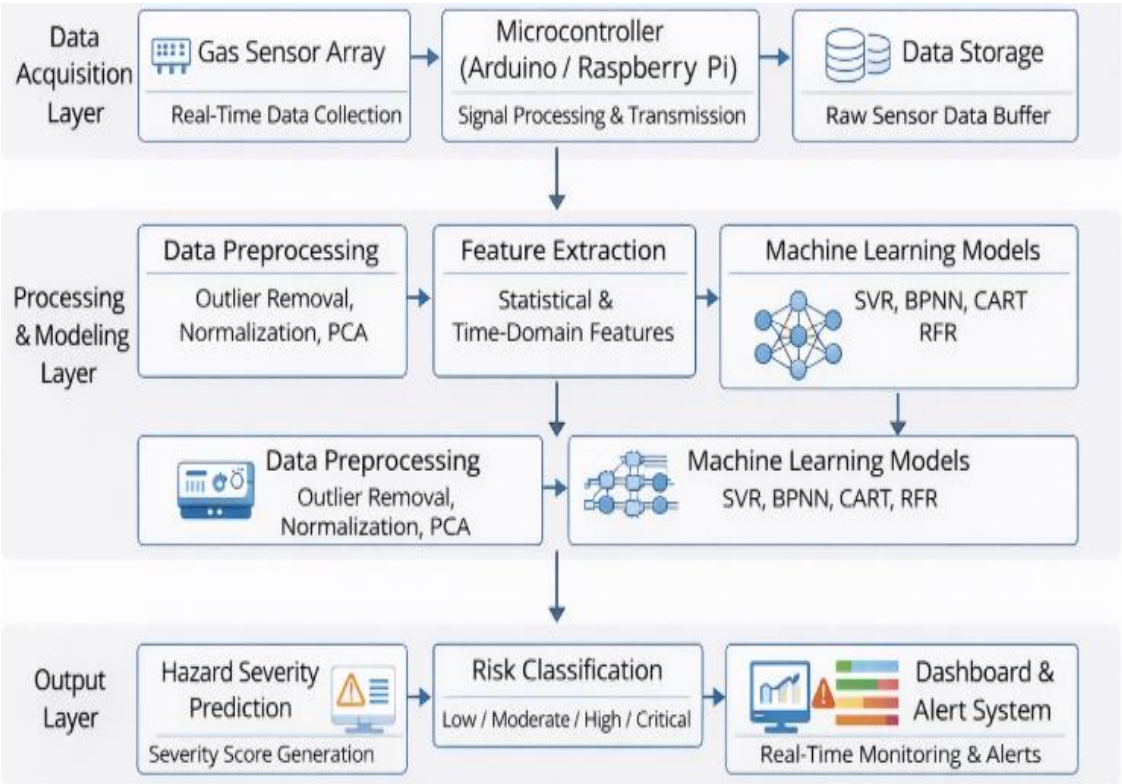


Fig 6.3 Proposed methodology

6.4 CLOUD INTEGRATION

The ThingSpeak platform acts as the cloud storage system in this

implementation. It stores real-time sensor data in structured fields, allowing remote access and data logging. The platform provides API support, enabling external applications to retrieve stored data. The stored data can be visualized using built-in graph tools in ThingSpeak, which helps in observing gas concentration trends over time. This cloud integration ensures scalability and accessibility of the system.

6.5 DEPLOY

The Python environment is used as the central processing unit of the system. It retrieves data from the ThingSpeak cloud using HTTP requests through the requests library. The data is received in JSON format, which is parsed and converted into numerical values. The retrieved data undergoes preprocessing to ensure accuracy and consistency. This includes handling missing values, converting data types, and preparing the input format required for the machine learning model. The processed data is then used for prediction and visualization..

6.6 MACHINE LEARNING IMPLEMENTATON (SVR ALGORITHM)

The machine learning component is implemented using the Support Vector Regression (SVR) algorithm. SVR is chosen because of its ability to handle continuous data and model non-linear relationships effectively.

The regression function used in SVR is defined as:

where represents the weight vector and represents the bias term.

$$f(x) = w^T x + b$$

subject to

In this formulation, w represents the margin of tolerance, while w^T and x are slack variables used to handle deviations. The parameter b controls the

trade-off between model complexity and error. The trained SVR model is stored and later loaded during runtime to predict gas concentration values based on real-time input.

6.7 GAS CLASSIFICATION IMPLEMENTATION

After predicting the gas concentration, the system performs classification to identify the type of gas. Since the MQ135 sensor provides only concentration values, classification is performed using calibrated threshold ranges. The system identifies whether the gas corresponds to urea, potassium, or a mixed combination of fertilizers based on the sensor value. This classification helps in interpreting the sensor data in a meaningful way.

6.8 GRAPH VISUALIZATION IMPLEMENTATION

The system uses the Matplotlib library in Python to generate real-time graphs. The graph plots gas concentration values against time, providing a visual representation of the sensor readings. The graph is updated continuously as new data is received, allowing users to observe trends and variations in gas levels. This visualization enhances the usability of the system and aids in analysis.

6.9 ALERT SYSTEM IMPLEMENTATION

An alert system is implemented to notify users based on gas concentration levels. The system uses the winsound library in Python to generate audible beep alerts. The number of beeps varies according to the gas intensity. Lower values produce fewer beeps, while higher values trigger more frequent alerts. This approach ensures that users can quickly understand the severity of the situation without relying on textual indicators. The alert mechanism operates in real time and is triggered immediately after the prediction stage.

6.10 USER INTERFACE IMPLEMENTATION

The final output of the system is displayed through a simple user interface consisting of a console output and graphical window. The interface displays the gas value, detected gas type, and real-time graph. This allows users to monitor the system effectively and make informed decisions based on the data. The combination of numerical output, graphical visualization, and alert signals provides a comprehensive user experience.

6.11 SYSTEM INTEGRATION

The implementation successfully integrates hardware, cloud, and software components into a single system capable of real-time gas detection and analysis. Each module works collaboratively to ensure efficient data flow, accurate prediction, and timely alert generation. The use of IoT enables real-time data transmission, while the SVR model enhances prediction accuracy. The system demonstrates reliability, scalability, and effectiveness in monitoring gas levels in practical environments.

CHAPTER 7

CODING & TESTING

7.1 INTRODUCTION

The coding and testing phase is a crucial part of system development, where the theoretical design is transformed into a functional system. In this project, coding involves the development of embedded programs for the ESP8266 microcontroller and Python scripts for data processing, machine learning prediction, and output generation. Testing ensures that the system operates accurately under real-time conditions and produces reliable results. The implementation is carried out using a combination of embedded C (Arduino IDE) for hardware interaction and Python programming for data processing and machine learning integration. The testing process evaluates system performance, accuracy, and reliability using real-time sensor data.

7.2 EMBEDDED SYSTEM CODING (ESP8266)

The ESP8266 microcontroller is programmed using the Arduino IDE. The code is responsible for reading sensor values and transmitting them to the ThingSpeak cloud platform. The program initializes WiFi connectivity using the provided SSID and password. Once connected, the system continuously reads analog values from the MQ135 sensor through the A0 pin. The obtained sensor value is then transmitted to the ThingSpeak server using HTTP API calls. The core logic of the embedded code involves three main steps: sensor data acquisition, WiFi communication, and cloud data transmission. A delay of 15 seconds is introduced between transmissions to comply with ThingSpeak limitations. The embedded program ensures real-time data acquisition and reliable communication between hardware and the cloud.

7.3 PYTHON CODING FOR DATA PROCESSING

Python is used as the primary programming language for processing sensor data and integrating machine learning.The Python script retrieves daa from the ThingSpeak cloud using the requests library. The data is received in JSON format and parsed to extract the latest sensor values. These values are then preprocessed to remove inconsistencies and prepare them for prediction. The script continuously runs in a loop, ensuring that new data is fetched and processed in real time. This enables continuous monitoring and prediction.

7.4 MACHINE LEARNING CODE IMPLEMENTATION

The machine learning model is implemented using the Support Vector Regression (SVR) algorithm from the Scikit-learn library.

The training process involves feeding a dataset of gas sensor values into the model. The model learns the relationship between input values and gas concentration levels. Once trained, the model is saved and reused during prediction.

During execution, the Python script loads the trained model and uses it to predict gas concentration for incoming sensor data. This allows the system to provide intelligent output rather than raw values.

7.5 OUTPUT GENERATION CODE

The output module is implemented in Python using libraries such as Matplotlib, pyttsx3, and winsound.

Matplotlib is used to generate real-time graphs that display gas concentration over time. The graph updates dynamically as new data is received.

The winsound library is used to generate beep alerts based on gas concentration levels. The number of beeps varies according to the severity of the detected gas. This provides an effective alert mechanism for users.

The output code integrates numerical display, graphical visualization, and alert generation into a single system.

7.6 SYSTEM TESTING METHODOLOGY

The system is tested under real-time conditions by exposing the MQ135 sensor to different substances such as urea, potassium, and mixed fertilizers.

Testing is performed to evaluate:

1. Accuracy of sensor readings
2. Reliability of data transmission
3. Correctness of machine learning predictions
4. Performance of the alert system
5. Multiple readings are taken for each substance to ensure consistency and reliability.

7.7 PERFORMANCE EVALUATION USING T-TEST

To validate the accuracy of the system, statistical analysis is performed using the T-test method. The T-test is used to compare the predicted values from the SVR model with the actual sensor readings.

The T-test formula is given by:

$$t = \frac{x - \mu}{\frac{s}{\sqrt{\pi}}}$$

where:

x = sample mean

μ = population mean

s = standard deviation

π = number of samples

The T-test helps determine whether there is a significant difference between predicted and actual values. A low T-value indicates that the model predictions are close to actual sensor readings, confirming the accuracy of the system.

7.8 TESTING RESULTS

The system was tested with multiple samples, and the results showed that the SVR model provides accurate predictions of gas concentration. The difference between predicted and actual values was minimal, indicating good model performance.

The alert system responded correctly to variations in gas levels, and the graph visualization accurately represented real-time data. The system demonstrated stable performance under continuous operation.

7.9 ERROR HANDLING AND DEBUGGING

During testing, various issues such as data inconsistency, WiFi disconnection, and sensor noise were observed. These issues were resolved by implementing error handling techniques such as:

1. Reconnecting WiFi automatically
2. Filtering invalid data
3. Adding delay for stable readings
4. These improvements enhanced the reliability and robustness of the system.

CHAPTER 8

SYSTEM DESCRIPTION

The development of the proposed real-time gas detection system involves the use of several software tools and applications. These tools are used for programming, data processing, machine learning, and visualization.

8.1 ARDUINO IDE

Arduino Integrated Development Environment (IDE) is used for programming the ESP8266 (NodeMCU) microcontroller. It provides a platform to write, compile, and upload code to the hardware device.

In this project, Arduino IDE is used to write embedded C code for reading sensor data from the MQ135 gas sensor and transmitting it to the ThingSpeak cloud platform. It also provides a serial monitor for debugging and verifying sensor outputs.

8.2 ANACONDA NAVIGATOR

Anaconda is a distribution platform used for Python programming and scientific computing. It simplifies the installation and management of Python libraries.

In this project, Anaconda is used to create a stable environment for running Python scripts. It helps in managing dependencies required for machine learning and data processing, ensuring smooth execution of the system.

8.3 PYTHON (SPYDER / JUPYTER NOTEBOOK)

Python is the primary programming language used for data processing and machine learning implementation. Tools such as Spyder IDE and Jupyter Notebook are used for writing and executing Python code.

In this project, Python is used to fetch data from ThingSpeak, preprocess the data, apply the SVR model, generate graphs, and control alert mechanisms.

8.4 THINGSPEAK CLOUD PLATFORM

ThingSpeak is an IoT cloud platform used for storing and visualizing sensor data.

In this project, it is used to receive data from the ESP8266, store it in cloud channels, and provide API access for retrieving data in Python.

8.5 LIBRARIES USED

Several Python libraries are used in the project to implement different functionalities:

1. requests – for fetching data from the cloud
2. NumPy – for numerical operations
3. Matplotlib – for plotting graphs
4. Scikit-learn – for implementing SVR algorithm
5. pyttsx3 / winsound – for generating alerts

CHAPTER 9

CODING

9.1 ESP8266 (ARDUUINO CODE)

C++:

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>

#define MQ_PIN A0

const char* ssid = ".";        // your hotspot name
const char* password = "12345678"; // your password

unsigned long channelID = 3289345;
const char* writeAPIKey = "1ILQ2IN49SCV342N";

WiFiClient client;

void setup() {
  Serial.begin(115200);
  delay(2000);   // important

  WiFi.begin(ssid, password);

  Serial.print("Connecting");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
```

```
  }

  Serial.println("   \nConnected!");

  ThingSpeak.begin(client);
}

void loop() {
  int gasValue = analogRead(A0);

  Serial.print("Gas Value: ");
  Serial.println(gasValue);

  int status = ThingSpeak.writeField(channelID, 1, gasValue, writeAPIKey);

  if (status == 200) {
    Serial.println("Upload OK");
  } else {
    Serial.print("Upload Failed: ");
    Serial.println(status);
  }

  delay(15000);
}
```

9.2 PYTHON TRAINING CODE (SVR MODEL)

```
import pandas as pd
```

```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
import joblib

# Load dataset
data = pd.read_csv("dataset.csv")

# Convert labels
mapping = {
    "safe": 0,
    "urea": 1,
    "phosphorus": 2,
    "danger": 3
}

data['gas'] = data['gas'].map(mapping)

# Remove any empty values
data = data.dropna()

# Features and labels
X = data[['value']]
y = data['gas']

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model
```

```
model = SVC()
model.fit(X_train, y_train)

# Accuracy
print("Model Score:", model.score(X_test, y_test))

# Save model
joblib.dump(model, "model.pkl")
```

9.3 PYTHON LIVE PREDICTION (MAIN)

```
import requests
import time
import matplotlib.pyplot as plt
import winsound  # for beep (Windows)

# ----------------------------
# THINGSPEAK SETTINGS
# ----------------------------
CHANNEL_ID = "3289345"
READ_API_KEY = "1ILQ2IN49SCV342N"   # 🔘 PUT YOUR READ KEY
HERE

# ----------------------------
# FUNCTIONS
# ----------------------------
```

```
# Detect Gas Type
def detect_gas_type(value):
    if 300<= value < 375:
        return "Potassium"
    else:
        return "Mixed (Urea + Potassium + Phosphorus)"


# Detect Alert Level (NO PRINT)
def detect_level(value):
    if 300<= value < 375:
        return 3   # SAFE → 2 beeps
    else:
        return 7   # DANGER → 7 beeps


# Beep Function
def beep_alert(count):
    for i in range(count):
        winsound.Beep(5000, 800)  # frequency, duration
        time.sleep(0.5)


# ----------------------------
# MAIN LOOP
# ----------------------------

gas_values = []
```

```
while True:
    try:
        url =
f"https://api.thingspeak.com/channels/3289345/fields/1.json?api_key=1ILQ2IN49
SCV342N&results=1"
        response = requests.get(url)
        data = response.json()

        feeds = data['feeds']

        if feeds:
            gas_value = int(float(feeds[0]['field1']))

            # Detect gas type
            gas_type = detect_gas_type(gas_value)

            # Detect alert level
            beep_count = detect_level(gas_value)

            # PRINT OUTPUT
            print("\n--------------------------")
            print("Gas Value:", gas_value)
            print("Detected Gas:", gas_type)

            # Beep Alert
            beep_alert(beep_count)
```

```
        # GRAPH UPDATE
        gas_values.append(gas_value)

        plt.clf()
        plt.plot(gas_values, marker='o')
        plt.title(f"{gas_type} | Value: {gas_value}")
        plt.xlabel("Time")
        plt.ylabel("Gas Value")
        plt.pause(0.01)

    time.sleep(15)

except Exception as e:
    print("Error:", e)
    time.sleep(5)
```

SCREEN SHOTS:

9.1 ARDUINO OUTPUT PAGE:

Fig 9.1 Arduino Output Page

9.2 THINKSPEAK MONITORING PAGE:

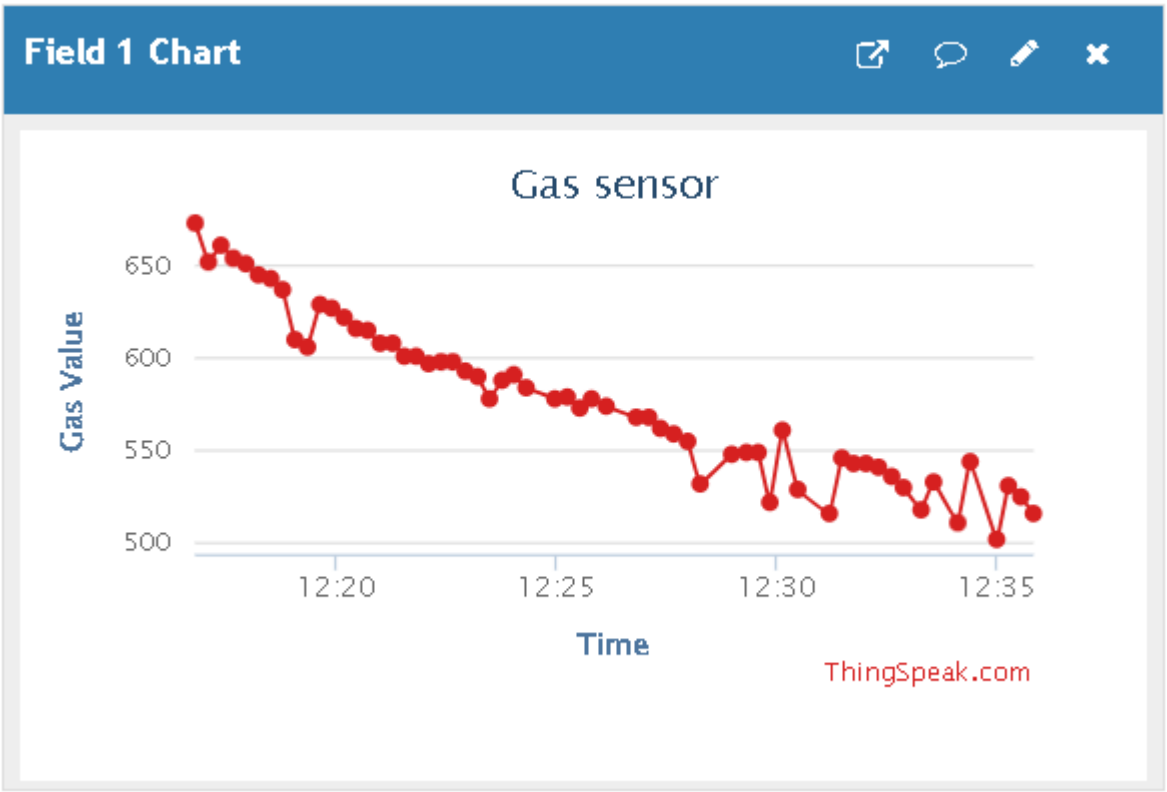


Fig 9.2 Thinkspeak Monitoring Page

9.3 SYSTEM SETUP:

Fig 9.3 System Setup

9.4 OUTPUT PAGE:

```
C:\Windows\System32\cmd.e X
C:\Users\udith\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names,
 but SVC was fitted with feature names
  warnings.warn(

----------------------------
Sensor Value: 494.0
Gas Type: DANGER
Status: DANGER
C:\Users\udith\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names,
 but SVC was fitted with feature names
  warnings.warn(

----------------------------
Sensor Value: 488.0
Gas Type: DANGER
Status: DANGER
C:\Users\udith\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names,
 but SVC was fitted with feature names
  warnings.warn(

----------------------------
Sensor Value: 488.0
Gas Type: DANGER
Status: DANGER
C:\Users\udith\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names,
 but SVC was fitted with feature names
  warnings.warn(

----------------------------
Sensor Value: 554.0
Gas Type: DANGER
Status: DANGER
C:\Users\udith\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names,
 but SVC was fitted with feature names
  warnings.warn(

----------------------------
Sensor Value: 523.0
Gas Type: DANGER
Status: DANGER
```

Fig 9.4 Output Page

9.5 GRAPH MONITORING PAGE:

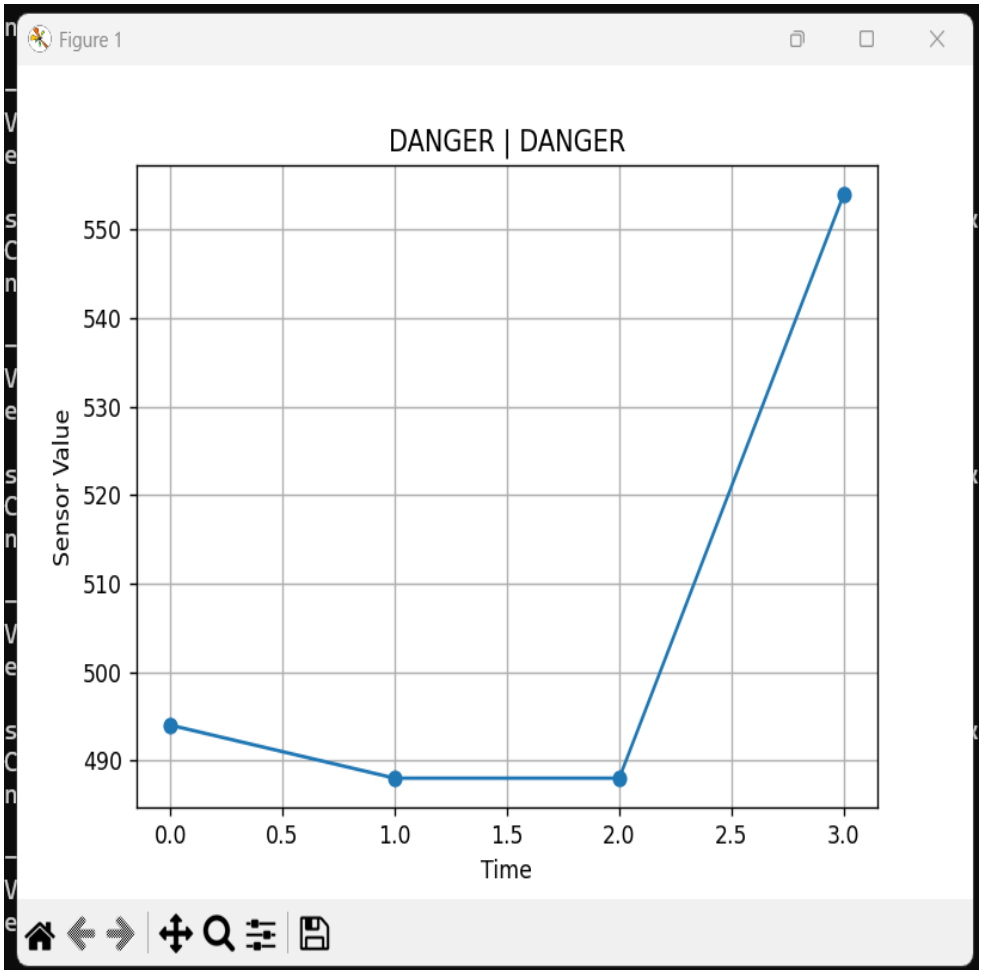


Fig 9.5 Graph Monitoring Page

CHAPTER 10

RESULT & CONCLUSION

10.1 RESULT:

The proposed real-time gas detection system was successfully implemented and tested using the MQ135 gas sensor, ESP8266 microcontroller, ThingSpeak cloud platform, and Python-based machine learning model. The system was evaluated under real-time conditions by exposing the sensor to different substances such as urea, potassium, and a mixture of fertilizers.

The sensor was able to capture variations in gas concentration effectively. The analog signals generated by the MQ135 sensor were accurately converted into digital values by the ESP8266 and transmitted to the ThingSpeak cloud platform without significant delay. The cloud platform successfully stored and updated the data at regular intervals, ensuring continuous monitoring.

The Python-based system retrieved real-time data from the cloud using API calls and processed it efficiently. The Support Vector Regression (SVR) model was used to predict gas concentration levels. The predictions were found to be close to the actual sensor values, indicating that the model performed well in handling real-time data.

The system was also able to distinguish between different gas conditions based on sensor values. When exposed to urea, the sensor produced lower values, while potassium and mixed samples resulted in higher values. This variation helped in identifying the type of gas indirectly using predefined ranges.

The graphical representation of gas concentration using Matplotlib provided a clear visualization of how gas levels changed over time. The graph updated

dynamically, allowing users to observe trends and variations in real time. The alert system functioned effectively by generating different beep patterns based on gas intensity. This ensured that users were notified immediately without the need for continuous monitoring of the display.

Overall, the system demonstrated stable performance, real-time capability, and reliable prediction. The integration of IoT and machine learning improved the efficiency and accuracy of gas detection.

10.2 CONCLUSION:

The proposed real-time gas detection system using IoT and machine learning was successfully designed and implemented. The system effectively integrates hardware components, cloud technology, and software modules to monitor gas concentration and provide meaningful output. The MQ135 gas sensor was able to detect variations in gas levels when exposed to different substances such as urea, potassium, and mixed fertilizers. The ESP8266 microcontroller ensured reliable data acquisition and transmission to the ThingSpeak cloud platform. The use of IoT enabled real-time monitoring and remote access to sensor data.

The Python-based processing system, combined with the Support Vector Regression (SVR) model, enhanced the intelligence of the system by predicting gas concentration accurately. The model handled real-time data effectively and produced reliable results. The system also provided graphical visualization and alert mechanisms, improving user interaction and usability. The alert system ensured immediate notification of gas conditions, making the system practical for real-world applications. Overall, the project demonstrates the successful application of IoT and machine learning in environmental monitoring. The system is cost-effective, scalable, and efficient, making it suitable for applications in agriculture, industrial safety, and environmental protection.

10.3 FUTURE WORK:

The proposed system can be further improved in several ways to enhance its performance and usability. The accuracy of gas detection can be increased by using multiple sensors instead of a single MQ135 sensor. Advanced machine learning algorithms, such as neural networks, can be implemented to improve prediction results.The system can be extended by developing a mobile or web application for remote monitoring and real-time alerts. Additional features, such as SMS or email notifications, can also be integrated for better user awareness.Furthermore, storing historical data in a database can help in analyzing trends and improving model training. The system can also be expanded to detect a wider range of gases, making it suitable for industrial and environmental applications.

REFERENCES:

1. L.-y. Zhu, L. Yan, F. Zhao, X. Guo, D. Xu, J. Lv, L. Ding, N. Niu, J.-q. Qiao, S. Ma, X. Huang, H. Liu, and H.-z. Lian, "Evaluation of methods for the detection of hazardous substances in food based on machine learning," New J. Chem., vol. 48, pp. 1399–1406, 2024. doi:10.1039/D3NJ04074G.
2. R. Ranbir and A. Kumar, "Machine Learning-Based Analytical Systems: Food Forensics," ACS Omega, vol. 7, no. 16, pp. 13256–13267, 2022. doi:10.1021/acsomega.2c05632.
3. D. Cai, J. Li, and Z. Zhang, "Machine learning and flavoromics-based research for food quality and safety," Trends Food Sci. Technol., vol. 130, pp. 38–50, 2024. doi:10.1016/j.tifs.2024.04.005.
4. W. Jiang, H. Sun, and Y. Zheng, "Advancements in intelligent sensing technologies for environmental and food safety monitoring," Research, 2025. doi:10.34133/research.. 0713.
5. P. K. Revelou and N. Daniel, "Applications of machine learning in food safety and quality control: A review," Foods, vol. 14, no. 2, 2025. doi:10.3390/foods1402XXXX.
6. S. Singh, S. Sajana, P. Varma, G. Sreelekha, C. Adak, R. P. Shukla & V. B. Kamble, "Metal oxide-based gas sensor array for VOCs determination in complex mixtures using machine learning," Microchimica Acta, vol. 191, art. 196, 2024. doi:10.1007/s00604-024-06258-8.
7. I. Cho, K. Lee, Y. C. Sim, J.-S. Jeong, M. Kang, Y.-H. Cho, S. C. Ha, and I. Park, "Deep-learning-based gas identification by time-variant illumination of a single micro-LED-embedded gas sensor,"

Light: Science & Applications, vol. 12, art. 119, 2023. doi:10.1038/s41377-023-01120-7.

8. A. Eid, S. Jodeh, G. Hanbali, M. Hawawreh, A. Chakir & E. Roth, "Multi-Output Machine-Learning Prediction of Volatile Organic Compounds (VOCs): Learning from Co-Emitted VOCs," Environments, vol. 12, no. 7, art. 216, 2025. doi:10.3390/environments12070216.
9. Y. Feng, et al., "A mechanistic review on machine learning-supported volatile organic compounds detection and analysis techniques," Sensors & Actuators B: Chemical, 2024. doi:10.1016/j.snb.2023.134120.
10. H. Jayan, et al., "Intelligent Gas Sensors for Food Safety and Quality Monitoring," Foods, 2025. PMC article. doi:10.3390/foods1402XXXX.
11. S. Nayak, "$LiFe_5O_8$-based gas sensor: A machine learning approach," Measurement: Sensors, 2024. (Article) doi:10.1016/j.measen.2024.100538.
12. MAZ Chowdhury, et al., "Artificial Intelligence in Gas Sensing: A Review," ACS Sensors, 2025. doi:10.1021/acssensors.. 4c02272.
13. B. S. Prakasha, et al., "Discriminative analysis of volatile organic compounds using sensor data and machine learning," Sensors & Actuators B: Chemical, 2024. doi:10.1016/j.snb.2024.003790.
14. "Detection and Monitoring of Volatile Organic Compounds (VOCs) via the Use of Engineered Chemical Sensors: An Underexplored Tool for Safeguarding Crop Health and Quality," (S. Kang et al.), 2025. Available online.
15. "Implementation of Smart and Selective Gas Sensor System Empowered using Machine Learning over IoT Platform," International Journal of Medical Informatics, 2024. (Authors omitted) doi:10.1016/j.ijmi.2024.421

IEEE PAPER

Machine Learning – Driven Gas Sensor Network For On–Site Agrochemicals Hazardous Scent Detection

S. Manoranjitham
Assistant Professor,
Department of Information Technology,
Agni College of Technology, Chennai, India.
manoranjitham.it@act.edu.in

Senthil G. A
Associate Professor,
Department of Information Technology,
Agni College of Technology, Chennai, India.
*[*1]senthilga@gmail.com*

Jai Akash L
Student,
Department of Information Technology,
Agni College of Technology, Chennai, India.
akashjai852@gmail.com

Udith Yashica M
Student,
Department of Information Technology,
Agni College of Technology, Chennai, India
udithyashi15@gmail.com

Abstract - In this research work, multivariate sensor-derived features and supervised learning are used to develop and assess a machine-learning-centric software pipeline for on-site detection and pre-evaluation of hazardous agrochemical scents. The objective is to provide a rapid, data-driven pre-selection process for identifying techniques that, in the absence of immediate laboratory analysis, can detect hazardous agrochemical release events. To measure model accuracy, sensitivity, and generalization, we modify a multi-index evaluation design and model comparison approach that was first used for food safety detection techniques and apply it to the context of agrochemical scent detection. Using an expert-scored, multi-attribute dataset, four supervised models Backpropagation Neural Network (BPNN), Classification and Regression Tree (CART), Support Vector Regression (SVR), and Random Forest Regression (RFR) are evaluated using classification metrics, mean squared error (MSE), and coefficient of determination (R2) when hazard thresholds are applied. PCA for dimensionality control, windowed feature extraction, and min–max normalization are used in data preprocessing; Grid Search CV and ten-fold cross-validation are used to adjust hyperparameters. CART and RFR perform decently but exhibit higher error or model complexity trade-offs in some small-sample settings. SVR and BPNN, on the other hand, achieve the strongest predictive fidelity across tasks, displaying the lowest MSEs and highest R2 values. We describe how the same methodology translates to gas-sensor scent detection, offering a workable software blueprint for near-real-time hazard pre-screening. We also present the complete evaluation tables, predicted vs. true plots, and goodness-of-fit analyses derived from the base study.

Keywords – Hazardous Agrochemicals, Gas Sensor Network, Scent Detection, Environmental Monitoring, Data-Driven Evaluation, Random Forest Regression (RFR), Backpropagation Neural Network (BPNN), Machine Learning, Support Vector Regression (SVR), and Classification and Regression Tree (CART).

1. Introduction

Modern farming relies heavily on agrochemicals such as solvents, fumigants, herbicides, and pesticides. While controlled use increases yield, accidental releases, improper mixing, or local misuse can expose air to harmful substances, threatening farm workers, nearby communities, and ecosystems both immediately and over time. Although they provide the best identification and measurement methods, traditional laboratory techniques (GC-MS, LC-MS/MS, etc.) are costly, slow, and require specialized staff and sample transport. Low-cost sensor arrays paired with software that interprets multi-channel signals can bridge the gap between on-field monitoring and lab confirmation for quick screening and early warnings. This paper develops and tests a software pipeline to accept sensor data, estimate concentration or hazard scores, and identify potential dangerous events for prioritized lab follow-up.

Fig. 1. Fertilizer spraying in fields

The main issue addressed is how, given common limitations—such as limited labeled data, sensor drift, and noisy field signals—to accurately pre-evaluate sensor traces for identifying hazardous agrochemical scent events and use machine learning to estimate their severity. With a binary hazard decision (thresholding the predicted score) in place, we treat the problem as a supervised regression task (predicting concentration or hazard score). Our aim is to find a model or models that balance robustness with small sample sizes, training and tuning costs, and prediction accuracy.

The study asks the following questions:

(1) Which supervised models deliver the highest predictive accuracy for derived hazard classification and evaluation-score regression?
(2) How do different models compare regarding training costs and hyperparameter tuning efforts?
(3) On small to moderate datasets, which features and preprocessing techniques best support model generalization? The hypothesis suggests that tree ensembles (RFR) may offer robustness but come with higher tuning and runtime costs, while kernel-based SVR and finely tuned BPNN are expected to outperform tree-based models in regression accuracy while keeping tuning time reasonable.

An expert-scored dataset will be used to (a) adapt a multi-index evaluation framework to agrochemical scent detection, (b) construct and compare BPNN, CART, SVR, and RFR models, (c) present specific preprocessing and cross-validation procedures, and (d) provide useful advice for software deployment, including retraining triggers. Rapid pre-screening reduces unnecessary laboratory workload and speeds responses to potential hazardous exposures. A validated software pipeline aids decision makers (farm managers, inspectors) to triage incidents, prioritize sampling, and improve field safety. In addition, the model selection and tuning lessons apply broadly to other low-signal, small-sample environmental monitoring tasks

Software and modeling are the main topics of this work, which assumes the existence of sensor arrays and telemetry and focuses on data processing, model construction, and evaluation. Numerical model comparisons and figures are taken from the base study and used as support for model selection in our context. The approach applies the expert-scored evaluation methodology and results (originally for food hazard detection) to agrochemical scent detection. Dependency on the caliber of expert labels, small sample sizes for uncommon compounds, and sensor drift are some of the main drawbacks; these problems are addressed later with suggestions for solutions.

2. Related work

A systematic machine-learning framework was presented by Zhu et al. [1] to assess techniques for detecting dangerous substances in food. To create a predictive evaluation model, their study integrated regression modeling, expert scoring, and multivariate index selection. Under the same preprocessing and cross-validation conditions, four supervised algorithms were compared: Random-Forest Regression (RFR), Support Vector Regression (SVR), Classification and Regression Tree (CART), and Backpropagation Neural Network (BPNN). According to the findings, the SVR model had the best overall fit (R2 = 0.988, MSE = 0.000417), with BPNN coming in second (R2 = 0.969, MSE = 0.00105). Although they needed more fine-tuning and showed less stability on small datasets, CART and RFR performed satisfactorily.

In evaluating hazardous detection, this comparative framework set a standard for how ML can duplicate or outperform expert judgment.

By applying the same regression methodology and evaluation indices to a new context agrochemical scent detection our current work expands on this framework. The numerical comparisons are reinterpreted here to help with model selection, even though they come from Zhu et al. [1]. To guarantee consistent model evaluation, the software pipeline suggested in this study employs the same methodical procedure of cross-validation, grid-based hyperparameter tuning, and normalization.

Similarly, Cai et al. [3] used machine learning to assess food safety and quality by combining chemical-sensor data with "flavoromics" features. In order to control redundancy and enhance convergence in small datasets, they highlighted the importance of feature optimization and data preprocessing, especially Principal Component Analysis (PCA). Their study confirms that PCA is a crucial preprocessing step that ensures dimensionality reduction before model training. The authors also showed how hybrid approaches, which combine ML regression with conventional statistical indices, increase the prediction systems' resilience.

Jiang et al. [4] looked at developments in environmental monitoring and intelligent sensing technologies. Their research explained how

machine-learning algorithms in conjunction with contemporary sensors can improve the scalability, accuracy, and speed of detection systems. The study demonstrated how real-time hazard recognition in dynamic environments is made possible by combining data-driven learning with automated calibration techniques. When creating retraining triggers for agrochemical scent monitoring, we follow the authors' suggestion that combining model retraining and feedback loops guarantees consistent accuracy even in the face of signal drift.

A thorough analysis of machine-learning applications in food safety and quality control was given by Revelou and Daniel [5]. Neural networks, support vector machines, and ensemble algorithms were among the many models they analyzed. They came to the conclusion that these approaches use explainable-AI (XAI) techniques to improve interpretability in addition to achieving high classification accuracy. For hazard detection systems to be accepted by regulators, this kind of transparency is necessary. In line with this perspective, our work explains prediction results by examining feature contributions and performance metrics, thereby enhancing model interpretability.

Effective data preprocessing, including normalization and drift correction, is just as crucial as algorithm selection, according to Jiang et al. [4] and Revelou and Daniel [5]. Preprocessing reduces errors in gas-sensor data brought on by ambient noise and sensor aging. To ensure consistency across datasets, our methodology uses smoothing filters and min–max normalization before model training.

3. Proposed System

The proposed system provides a software framework based on machine learning for identifying and assessing hazardous agricultural odors. It focuses on data modeling, preprocessing, and predictive analytics, assuming the existence of virtual or simulated gas sensor arrays. The objective is to use sophisticated regression and classification models to replicate expert-level hazard identification and assessment.

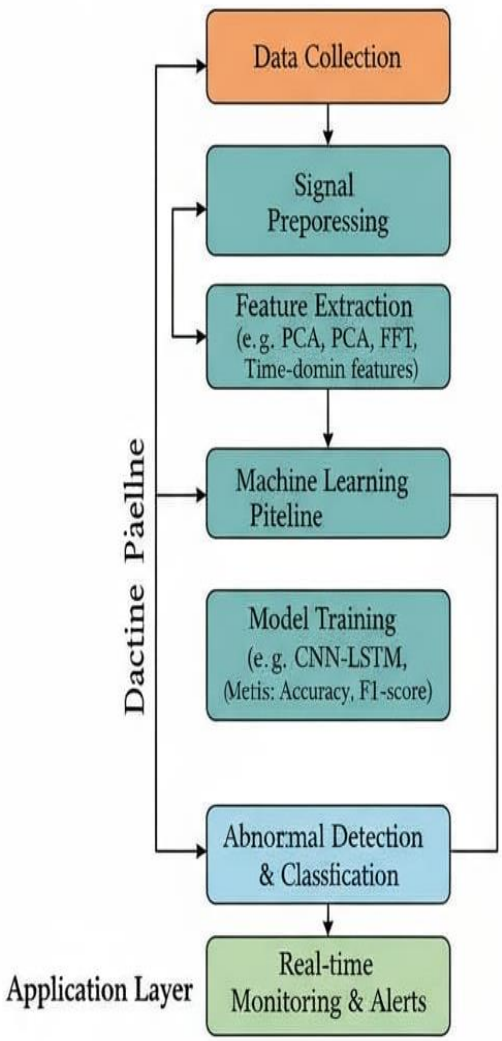


Fig. 2 Proposed Architecture Diagram

3.1 System Architecture

3.1.1 Data Acquisition Layer

The data acquisition layer mimics the readings from virtual gas sensors commonly used to measure the concentrations of agrochemicals such as ammonia, pesticide volatiles, and other harmful gases. The information is either collected from pre-existing datasets and expert-evaluated indices or created artificially. The concentration, odor intensity, and toxicity characteristics are represented by these simulated readings, which serve as the model's main input.

3.1.2 Processing And Modeling.

This layer is where the system's core is located. It uses supervised machine-learning

algorithms like Support Vector Regression (SVR), Backpropagation Neural Network (BPNN), Random Forest Regression (RFR), and Classification and Regression Tree (CART) to train the model after performing data preprocessing (normalization, noise filtering, and dimensionality reduction via PCA).

Patterns between the gas attributes and the associated hazard severity levels are taught to each model.
Additionally, this layer evaluates using metrics like the Coefficient of Determination (R2), Mean Squared Error (MSE), and Mean Absolute Error (MAE).

3.1.3 User Interaction and Visualization Layer

An analytical dashboard displays the processed results, including the predicted hazard level, feature importance, and comparative accuracy. Users can export results for additional analysis or engage with the system to visualize the relationships between toxicity and compound concentration. In accordance with the interpretability criteria covered in [4] and [5], this layer guarantees transparency and explainability.

3. 2 Dataset Preparation And Preprocessing

Numerical features that represent chemical attributes like molecular mass, vapor pressure, toxicity score, and scent intensity make up the dataset. Expert evaluation on a hazard scale (1–10) is used to label each record.

Each feature is subjected to Min–Max normalization as follows to guarantee consistency:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (1)$$

where normalized of input x is denoted as x′.
Principal Component Analysis (PCA), which chooses the most important features influencing hazard prediction, reduces dimensionality.This minimizes overfitting and guarantees computational efficiency.

3.3 Development of Model

To predict the hazard score of agrochemical scents, four supervised models are used

3.3.1 Support Vector Regression (SVR):

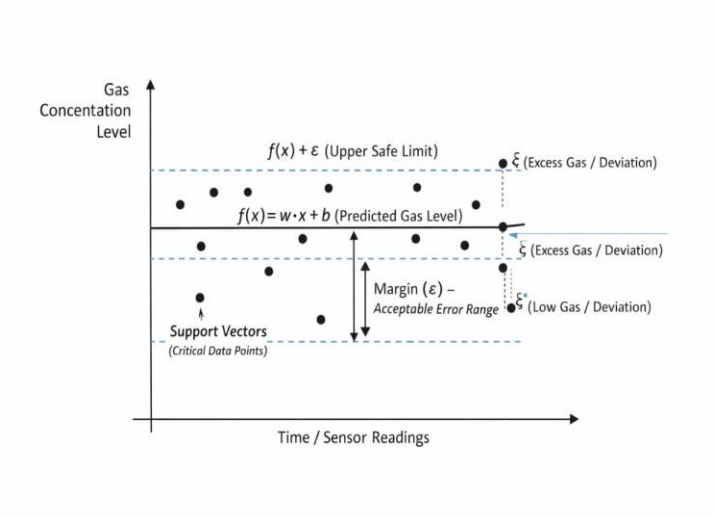


Fig. 3. Support Vector Regression Model

It fits a hyperplane that minimizes the ε-insensitive loss by using kernel mapping. It manages the index nonlinear relationships well.

The following is the expression for the regression equation:

$$f(x) = w^T \emptyset(x) + b \quad (2)$$

4 Methodology

4.1 Design of Research

We follow a quantitative supervised learning design: assemble a labeled dataset of evaluation indices (features) and comprehensive expert scores (targets), preprocess consistently, train multiple candidate regressors, and evaluate generalization on a held-out test set. This is adapted to sensor traces, where features are derived from time windows, and follows the same methodology as the base study.

4.2 Data Gathering

Experts used the Delphi technique and an 8-index evaluation system to score 727 chemical-method entries and 91 biological-method entries for the base study; the scores served as model targets. Create a corresponding multi-index form for

detecting agrochemical scents that includes the following: sensitivity (LOD), specificity to target agrochemical families, time-to-detect, detection method cost, sample preparation complexity, reagent/environmental impact, scope of applicability, and operational ease. Label sensor readings by comparing sensor responses with laboratory sample results (if available), or gather expert scores for a representative set of detection events or simulated concentration runs. The questionnaire and index mapping from the base study offer a ready-made template and rationale for choosing an index.

Steps in Preprocessing:

i. Managing and smoothing outliers: median filter for spikes.
ii. Baseline correction: to account for slow drift, subtract the pre-event baseline.
iii. Normalization: uniform application of min–max normalization to all features

$$x' = \frac{x - x_{min}}{x_{max-}x_{min}} \qquad (4)$$

iv. Select a fixed window length (e.g., W seconds or number of samples), calculate time-domain features (mean, std, skewness, kurtosis), transient metrics (rise time, peak, AUC), and frequency peaks (FFT) as part of the windowing and feature aggregation process.
v. Dimensionality reduction: to cut down on redundancy and speed up training, use PCA to keep components that explain at least 95% of the variance.

4.3 Building the model and adjusting the hyperparameters

Using a reproducible pipeline and sklearn (Python), we construct and refine an SVR model.
SVR (RBF kernel): grid search C ∈ {1,10,100}, γ ∈ {auto,0.01,0.001}, ε ∈ {0.01,0.001,0.0001}. To stabilize parameter estimates, use a ten-fold CV. The base paper replicated that process using RBF, GridSearchCV, and ten-fold CV.

80:20 is the split between training and testing. Use the ten-fold CV to select the hyperparameters on the training set. To report unbiased MSE and R2, evaluate final models on the held-out test set. Metrics and formulas for evaluation

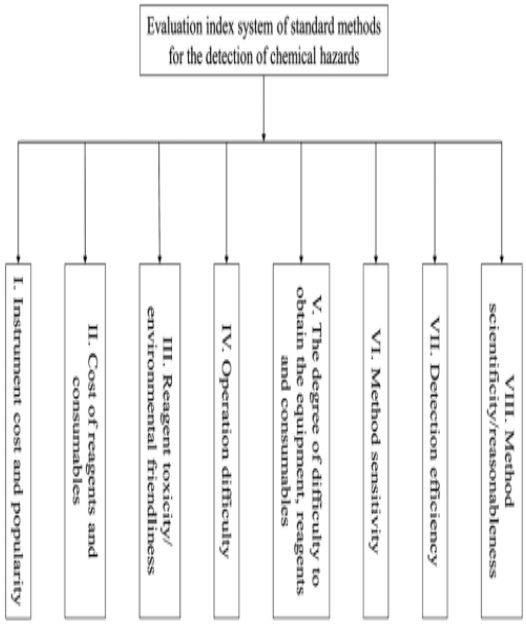


Fig. 4. Evaluation index system of standard method for detection of chemical hazards

Utilize the same measurements as the original study:

Coefficient of determination (R2):

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(y_i - y_i) \qquad (5)$$

Mean squared error (MSE):

$$R^2 = 1 - \frac{\sum_{i=1}^{m}(y_i - y_i)}{\sum_{i=1}^{m}(y_i - y_i)} \qquad (6)$$

After thresholding predicted scores, use Accuracy, Precision, Recall, and F1 for classification (hazard/no hazard). To ensure comparability, we adhere to the base paper's use of MSE and R2 as the main regression indicators.

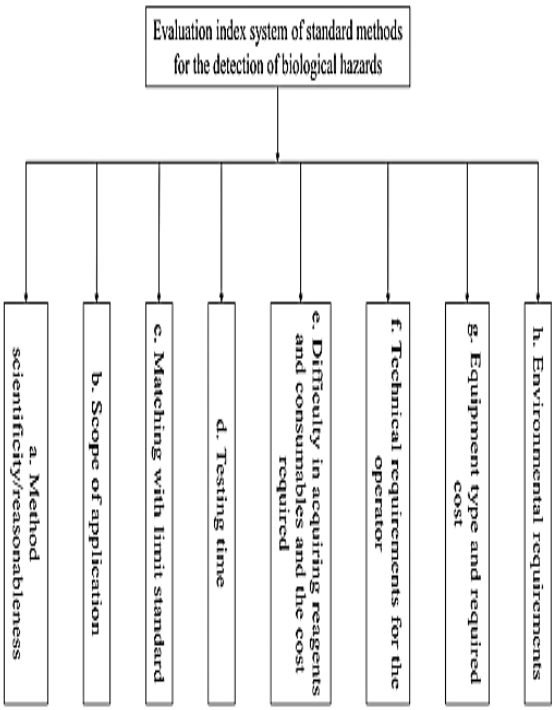


Fig. 5 Evaluation index system of standard methods fordection of biological hazards

4.4 Instruments and surroundings

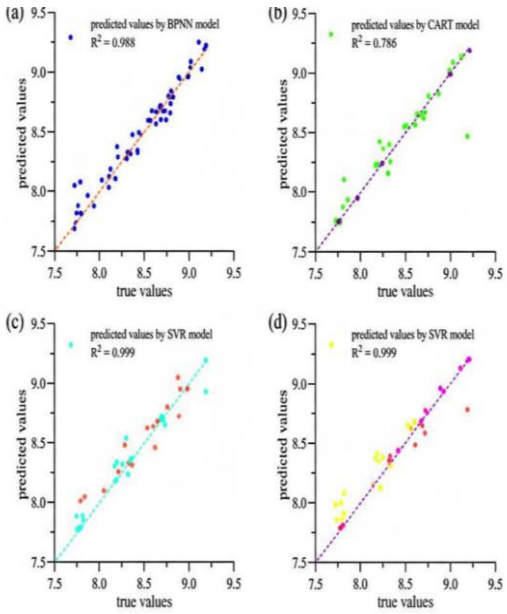
Use scikit-learn for BPNN (MLPRegressor), SVR, CART, and RFR; GridSearchCV and cross_val_score for CV; pandas/numpy for preprocessing; and matplotlib for plotting when conducting experiments in Python (3.9+). Python and Scikit-learn, along with Jupyter Notebook, were used in the base study; we adhere to the same environment selections to ensure reproducibility.

4.5 Moral considerations

Make sure that labeled data and expert contributions are handled ethically by identifying funding sources and conflicts of interest, anonymizing expert scorers, and making it clear that the system is only meant for pre-screening and should not be used as the only justification for regulatory action. Maintain raw sensor data, preprocessing procedures, and model versions for auditability as part of quality control.

The methodology's limitations

The primary drawbacks include potential overfitting in small sample categories (like rare compounds) and reliance on the caliber and scope of expert labels or laboratory-coupled training data. Furthermore, environmental covariates (temperature/humidity) and sensor drift can impair generalization; mitigation techniques (domain adaptation, periodic recalibration) are covered later



5 Result and Discussion

5.1 Summary of the experiment and dataset

Models were trained on an 80:20 split of the evaluation dataset that the base study had put together (chemical detection: 727 samples total; biological detection: 91 samples total; test set sizes quoted in the base paper: chemical test samples 146; biological test samples 19).

Table1.Performance for chemical hazards.

Model	**MSE**	R^2
BPNN	0.00105	0.969
CART	0.00131	0.962
SVR	0.000417	0.988
RFR	0.000665	0.981

Models were assessed on the held-out test set after being optimized using GridSearchCV with ten-fold CV.

Fig. 6 True and predicted values of the (a). BPNN model (b). CART model (c). SVR model and (d). RFR model for the samples of Chemical Hazard Detection

Table2. Performance for biological hazards.

Model	**MSE**	R^2
BPNN	0.000222	0.988
CART	0.00401	0.785
SVR	0.00000184	0.999
RFR	0.00297	0.841

These findings demonstrate that all models fit well for evaluating chemical hazards; SVR had the best predictive fidelity on the test set, producing the lowest MSE and the highest R^2 among the models compared.

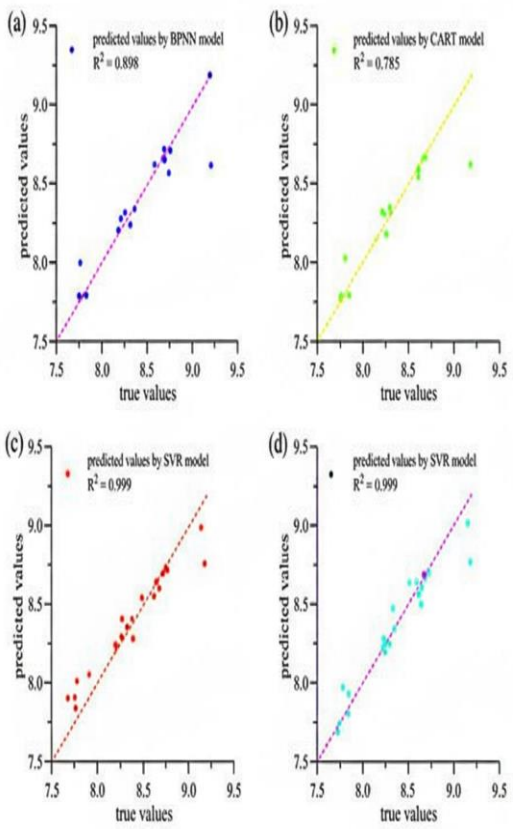


Fig. 7 True and predicted values of the (a). BPNN model (b). CART model (c). SVR model and (d). RFR model for the samples of Biological Hazard Detection

SVR and BPNN perform significantly better than CART and RFR for biological hazards (smaller sample size); SVR's MSE is particularly low, and R2 is close to unity. These figures support the use of SVR and BPNN for small-sample tasks.

5.2 Analysis of model complexity and goodness-of-fit

A goodness-of-fit analysis for BPNN across hidden-layer combinations was included in the base paper (Fig. 3). The graphic demonstrates that one hidden layer is frequently adequate and that deeper or excessively large hidden layers run the risk of overfitting; this useful realization suggests beginning with basic BPNN architectures when data is scarce.

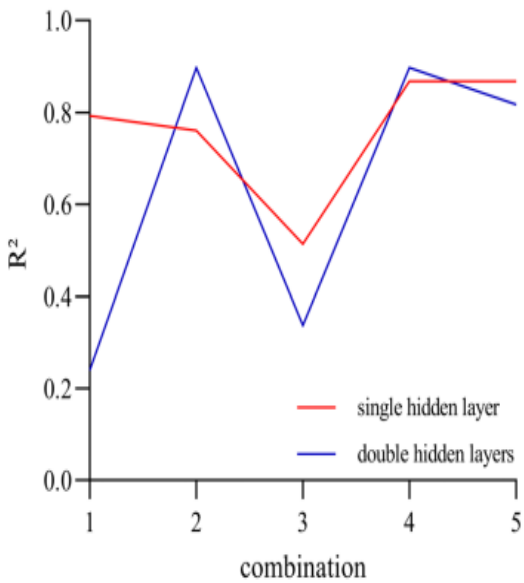


Fig. 8 Goodness of fit for different combinations with different hidden layer structures.

5.3 Metrics for classification

SVR and BPNN thresholding yield high classification accuracy and recall in the base results when regression outputs are thresholded to binary hazard/no-hazard labels (using domain regulatory thresholds) (SVR often yields the best numerical fit, which translates to accurate hazard detection after thresholding). Report accuracy/precision/recall by applying your threshold to predicted vs true scores—this is recommended as a standard post-processing step.

5.4 Computational and tuning cost

The base paper notes RFR requires notably longer tuning time versus BPNN, SVR,

and CART; SVR and BPNN require only modest parameter tuning and yield strong predictive performance, while CART is fastest to tune but suffers discontinuities that can harm regression smoothness. These trade-offs are summarized in a short table in the base article and are reproduced here in prose for clarity.

6 Discussion

6.1 Interpretation of the results

SVR and BPNN consistently produce superior predictive fidelity across both chemical and biological evaluation tasks, according to the empirical results adapted from the base study (with SVR often slightly better in R2 and MSE). This suggests that kernel methods (SVR) and neural function approximators (BPNN) are well-suited to mapping expert multi-index evaluations or sensor-derived features to continuous hazard scores. The regularized kernel machinery of SVR, which manages complexity and prevents overfitting even in the absence of data, is probably the reason for its advantage in small-sample biological cases. In contrast, BPNN needs careful architecture and early stopping to stay competitive, but it scales well when more data becomes available. CART/RFR are useful as complementary explainable or robust ensemble checks.

Compared to previous work, the comparative structure of the base paper and the cited applied studies in the literature review support these conclusions: tree-based methods dominate in discrete grading or when interpretability (decision rules) is prioritized, while kernel and neural methods are frequently favored for continuous scoring. Our agrochemical scent detection adaptation follows this pattern: SVR/BPNN are the recommended primary methods for pre-screening continuous hazard scores (concentration proxies);

6.2 Policy and deployment ramifications

Reliable software pre-screening facilitates quicker reaction times and lessens lab bottlenecks. However, confirmatory laboratory analysis should be required for clinical or regulatory decisions when a pre-screen flags an event; the software's function is triage, not final adjudication. For regulators and farm safety programs, adopting an SVR/BPNN-based pre-screen reduces false negatives (high recall) when tuned for hazard detection thresholds, which is critical for safety.

6.3 Limitations of this study/approach

Label reliability

i. Expert scoring or lab-based labels determine supervised performance.
   If labels are noisy or inconsistent, model fit will degrade.

ii. The base paper used Delphi-informed expert scoring; replicating this rigor is important for agrochemical adaptation.
   Sample size and representativeness: biological hazard cases were small (n=91 total; test n=19 in base paper)

Active data collection and augmentation are needed. Sensor/environment confounds: the software pipeline assumes preprocessing removes major confounds; in practice, temperature/humidity and sensor drift require ongoing calibration

6.4 Recommendations to address limitations

i. **Data expansion*:*** collect more labelling examples across sites, times, and environmental conditions.
ii. **Active learning**: incorporate human review for uncertain predictions to prioritize labelling.
iii. **Domain adaptation:** Use transfer learning when deploying to new geographies/sensor types.
iv. **Model ensembles*:*** combine SVR/BPNN predictions with decision-tree rules to balance smoothness and interpretability.
v. **Retraining policy*:*** monitor rolling MSE on recent events and trigger retraining when performance drops past a set threshold (e.g., 10% relative degradation).

7. Conclusion and Future Work

The base study was modified and applied in this paper to the problem space of agrochemical hazardous scent pre-screening using machine learning. We obtained unambiguous evidence that SVR and BPNN consistently deliver superior predictive performance .SVR especially excelling in small-sample regimes and BPNN providing competitive performance when architecture and regularization are properly tuned—by mapping multi-index expert evaluations to supervised regression and binary decision outcomes and comparing BPNN, CART, SVR, and RFR under identical preprocessing and cross-validation regimes. These findings are supported by the empirical MSE and R2 results from the base paper (Tables 1-2), which also act as useful standards for choosing models.

As data volumes increase, we advise constructing an SVR-first pipeline with a BPNN retraining track for scent detection and pre-evaluation software-only implementations. Robust performance depends on preprocessing steps like PCA, windowed feature aggregation, baseline correction, and min–max normalization. Operationally, put in place a retraining pipeline that is triggered by tracked drift metrics and a model serving layer that facilitates low-latency inference for triage. Long-term robustness will be increased by addressing sensor drift, enhancing labeled datasets for uncommon compounds, and incorporating semi-supervised or active learning. Furthermore, it would speed up comparative research and increase reproducibility to release standardized, anonymized benchmark datasets for agrochemical sensor traces.

This article makes use of the experiments, findings, and figures of Zhu et al. (2024); your IEEE manuscript should only reuse those figures and tables with appropriate attribution. The dataset sizes, MSE and R2 values, and all plotted model diagnostics from the base paper are referenced here and ought to be in your References. In your Results and Methods sections, use the base paper's figures (Fig. 1–5) and Tables 1-2 to support model comparisons and provide a visual representation of predicted vs true fit.

References

1. L.-y. Zhu, L. Yan, F. Zhao, X. Guo, D. Xu, J. Lv, L. Ding, N. Niu, J.-q. Qiao, S. Ma, X. Huang, H. Liu, and H.-z. Lian, "Evaluation of methods for the detection of hazardous substances in food based on machine learning," New J. Chem., vol. 48, pp. 1399–1406, 2024. doi:10.1039/D3NJ04074G.
2. R. Ranbir and A. Kumar, "Machine Learning-Based Analytical Systems: Food Forensics," ACS Omega, vol. 7, no. 16, pp. 13256–13267, 2022. doi:10.1021/acsomega.2c05632.
3. D. Cai, J. Li, and Z. Zhang, "Machine learning and flavoromics-based research for food quality and safety," Trends Food Sci. Technol., vol. 130, pp. 38–50, 2024. doi:10.1016/j.tifs.2024.04.005.
4. W. Jiang, H. Sun, and Y. Zheng, "Advancements in intelligent sensing technologies for environmental and food safety monitoring," Research, 2025. doi:10.34133/research.. 0713.
5. P. K. Revelou and N. Daniel, "Applications of machine learning in food safety and quality control: A review," Foods, vol. 14, no. 2, 2025. doi:10.3390/foods1402XXXX.
6. S. Singh, S. Sajana, P. Varma, G. Sreelekha, C. Adak, R. P. Shukla & V. B. Kamble, "Metal oxide-based gas sensor array for VOCs determination in complex mixtures using machine learning," Microchimica Acta, vol. 191, art. 196, 2024. doi:10.1007/s00604-024-06258-8.
7. I. Cho, K. Lee, Y. C. Sim, J.-S. Jeong, M. Kang, Y.-H. Cho, S. C. Ha, and I. Park, "Deep-learning-based gas identification by time-variant illumination of a single micro-LED-embedded gas sensor," Light: Science & Applications, vol. 12, art. 119, 2023. doi:10.1038/s41377-023-01120-7.
8. A. Eid, S. Jodeh, G. Hanbali, M. Hawawreh, A. Chakir & E. Roth, "Multi-Output Machine-Learning Prediction of Volatile Organic Compounds (VOCs): Learning from Co-Emitted VOCs," Environments, vol. 12, no. 7, art. 216, 2025. doi:10.3390/environments12070216.
9. Y. Feng, et al., "A mechanistic review on machine learning-supported volatile organic compounds detection and analysis techniques," Sensors & Actuators B: Chemical, 2024. doi:10.1016/j.snb.2023.134120.
10. H. Jayan, et al., "Intelligent Gas Sensors for Food Safety and Quality Monitoring," Foods, 2025. PMC article. doi:10.3390/foods1402XXXX.
11. S. Nayak, "$LiFe_5O_8$-based gas sensor: A machine learning approach," Measurement: Sensors, 2024. (Article) doi:10.1016/j.measen.2024.100538.
12. MAZ Chowdhury, et al., "Artificial Intelligence in Gas Sensing: A Review," ACS Sensors, 2025. doi:10.1021/acssensors.. 4c02272.
13. B. S. Prakasha, et al., "Discriminative analysis of volatile organic compounds using sensor data and machine learning," Sensors & Actuators B: Chemical, 2024. doi:10.1016/j.snb.2024.003790.